

Notes on Shuffle/Exchange-Type Switching Networks

D. STOTT PARKER, JR., MEMBER, IEEE

LIBRARY
LINK DIVISION
SINGER COMPANY
BINGHAMTON, N. Y. 13902

Abstract—In this paper a number of properties of Shuffle/Exchange networks are analyzed. A set of algebraic tools is developed and is used to prove that Lawrie's inverse Omega network, Pease's indirect binary n -cube array, and a network related to the 3-stage rearrangeable switching network studied by Clos and Beneš have identical switching capabilities. The approach used leads to a number of insights on the structure of the fast Fourier transform (FFT) algorithm. The inherent permuting power, or "universality," of the networks when used iteratively is then probed, leading to some nonintuitive results which have implications on the optimal control of Shuffle/Exchange-type networks for realizing permutations and broadcast connections.

Index Terms—Fast Fourier transform (FFT) algorithm, indirect binary n -cube, interconnection switches, Omega network, network universality, 3-stage rearrangeable switching network, Shuffle/Exchange.

I. INTRODUCTION

FOR SOME time it has been known that Shuffle/Exchange networks provide an effective interconnection scheme for parallel computation on many problems—see, for example, [15] and [7]. These networks are constructed of repeated copies (or a cyclically reused single copy) of a "perfect shuffle" connection followed by a column of 2×2 crosspoint elements which can exchange adjacent line values independently (see Fig. 1). Stone points out in [15] that this network can be used for sorting using a method described by Batcher [1], evaluating polynomials, transposing matrices, and computing fast Fourier transforms (FFT's) (as Pease showed [10]); Lawrie, in [7], shows that many useful routing permutations (in particular those arising in matrix computations) can be realized using these networks; and others (e.g., [5], [6]) have shown that networks based on the simple Shuffle/Exchange have other interesting properties.

The standard model for a multistage (= multicopy) Shuffle/Exchange network is the *Omega network* of Lawrie, which consists of $n = \lg(N)$ Shuffle/Exchanges,¹ where N is the number of input lines (which we will assume is a power of 2). The Omega network for $N = 16$ is illustrated in Fig. 1.

Manuscript received July 3, 1978; revised June 10, 1979. This work was supported by the National Science Foundation under Grants MCS76-81686 and MCS75-21758 and by an IBM Foundation.

The author was with the Department of Computer Science, University of Illinois, Urbana-Champaign, IL 61801. He is now with Department of Computer Science, University of California, Los Angeles, CA 90024.

¹ $\lg(x) = \log_2(x)$.

The purpose of this paper is to compare properties of this network with those of two other networks: first, Pease's *indirect binary n -cube array* [11] (Fig. 2), and second, a network obtained by appending a bit-reversal connection—clarified later on—to the first half of the rearrangeable switching networks (RSN's) discussed by Clos, Beneš, and others [2], which will be referred to in this paper as an *R-network* (Fig. 3). We define the *inverse Omega network* to be precisely what we would get if we ran the Omega network "backwards," i.e., let data flow from right to left in Fig. 1, instead of left to right. Thus, the inverse Omega network is just $\lg(N)$ Exchange/Unshuffles. (Unshuffles are often called inverse perfect shuffles.) With this definition we can make the following statement:

The inverse Omega network, the indirect binary n -cube, and the *R-network* are all equivalent.

The word "equivalent" may be interpreted validly in at least two different ways (topological equivalence or functional equivalence), and in fact results about the equivalence of algorithms may be derived from the statement if one views the networks as operating on the input data in ways other than just permuting it.

The above statement is not a new result, having been observed by both Pease [11] and Siegel [14], as well as others. We will prove it in the next section to show the effectiveness of new algebraic tools for studying these networks. This result is useful since it lets us apply what we know about any one of the networks to the others. In particular, the proof and algebraic tools are used here to get a better understanding of the FFT algorithm. The standard FFT, with a "butterfly" algorithm graph related to the indirect binary n -cube network, is transformed into Pease's shuffle-based algorithm [9], and then transformed into an algorithm, based on the *R-network*, without the usual bit-reversal phase: the FFT outputs are produced in correct order.

Having established firmly this network equivalence we then address the topic of the "universality" of these and other networks (their ability to realize arbitrary permutations if multiple passes through them are permitted). This question has been examined in [13] and [14]. Here it is more fully developed, and the results are shown to have interesting implications on the problem of controlling Shuffle/Exchange-type networks to realize arbitrary permutations or to sort data in parallel.

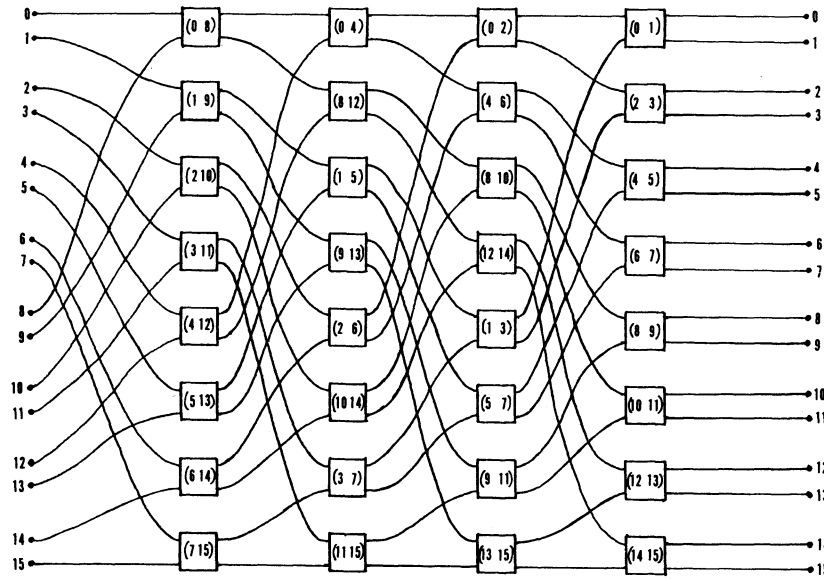


Fig. 1. A 16-input Omega network.

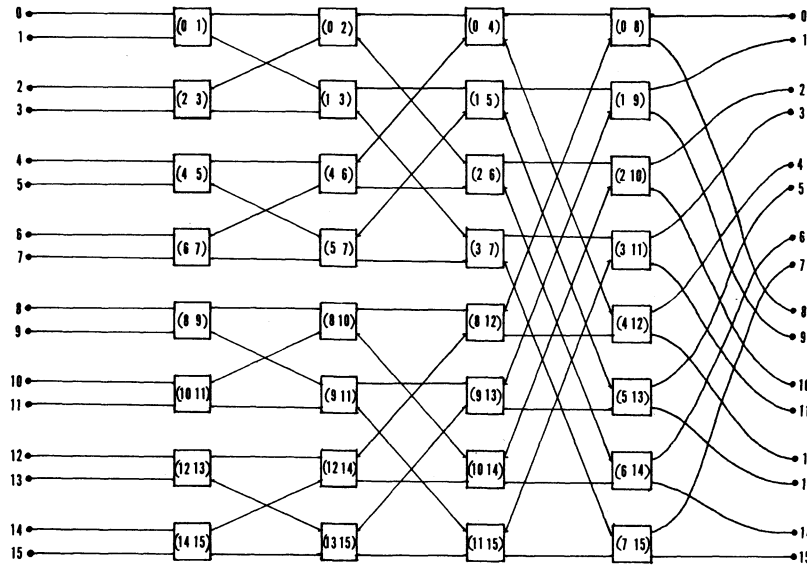


Fig. 2. An indirect binary 4-cube network.

II. FUNDAMENTAL CONNECTIONS AND FORMAL SWITCH DEFINITIONS

In this section we will be concerned only with networks having $N = 2^n$ inputs and outputs, where n is an integer. Although the results presented here generalize for the case where N is not a power of 2, we content ourselves for the time being with this restriction. We now define 4 basic permutations which suffice to generate the Omega, indirect binary n -cube, and R -networks—the shuffle (σ), butterfly (β), bit reversal (ρ), and exchange (E) permutations.

The *perfect shuffle permutation* σ is defined by

$$\sigma(x) = (2x + \lfloor 2x/N \rfloor) \bmod N$$

where x , the index of some input line (i.e., its order from the top of all N lines), lies between 0 and $N - 1$ (see Fig. 1). Perhaps a more cogent way of describing this is to say that if

$x = [x_n x_{n-1} \cdots x_1]$ when x is described in binary notation (so $x = (2^{n-1})x_n + \cdots + (2)x_2 + x_1$) then shuffling corresponds to a circular left shift of the index bits:

$$\sigma(x) = \sigma([x_n x_{n-1} \cdots x_1]) = [x_{n-1} x_{n-2} \cdots x_1 x_n]. \quad (1)$$

Thus it is clear that σ^{-1} , the unshuffle, corresponds to a circular right shift. We also define the k th *subshuffle* $\sigma_{(k)}$, for $1 \leq k \leq n$, by

$$\sigma_{(k)}(x) = \sigma_{(k)}([x_n \cdots x_1]) = [x_n \cdots x_{k+1} x_{k-1} \cdots x_1 x_k].$$

So $\sigma_{(k)}(x)$ is a shuffle on the k least significant bits in x 's binary representation, $\sigma_{(1)}(x) = x$, and clearly $\sigma_{(n)} = \sigma$.

Using this notation we define the *bit reversal permutation* ρ by

$$\rho([x_n x_{n-1} \cdots x_2 x_1]) = [x_1 x_2 \cdots x_{n-1} x_n] \quad (2)$$

and the k th *butterfly permutation* $\beta_{(k)}$, for $1 \leq k \leq n$, by

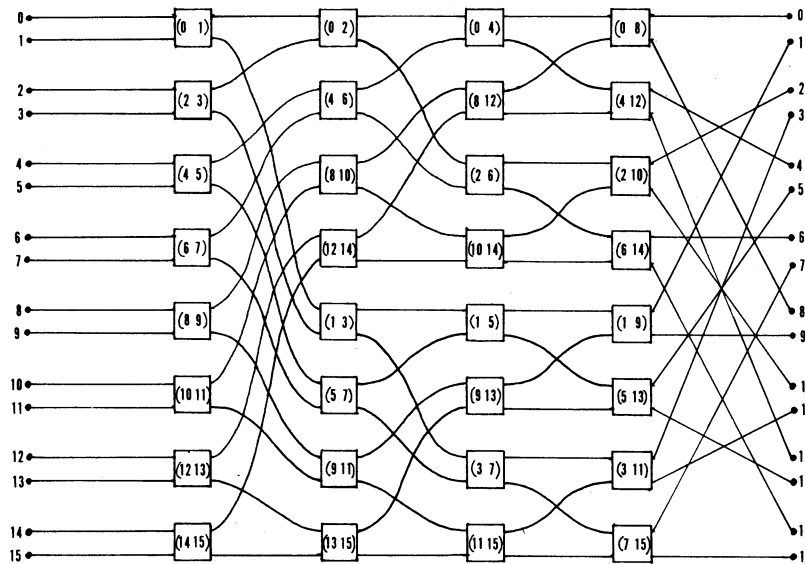


Fig. 3. A 16-input R-network.

$$\beta_{(k)}([x_n \cdots x_1]) = [x_n \cdots x_{k+1} x_1 x_{k-1} \cdots x_2 x_k], \quad (3) \quad \text{Then}$$

i.e., the butterfly permutation interchanges the first and k th bits of the index. Also, if we define $\hat{x}$ for all $0 \leq x \leq N-1$ with $x = [x_n \cdots x_1]$ by

$$\hat{x} = [x_n \cdots x_2 \bar{x}_1]$$

where the bar indicates Boolean complementation, then we can say that the set E of exchange permutations is

$$E = \{\text{permutations } e \mid \text{for every } 0 \leq x \leq N-1 \text{ with } x \text{ even} \\ \text{either } e(x) = x \text{ and } e(\hat{x}) = \hat{x} \\ \text{or } e(x) = \hat{x} \text{ and } e(\hat{x}) = x\}. \quad (4)$$

Note: E is a set with $2^{N/2}$ elements.

The immediate application of these definitions is that they describe the permutations that can be realized using the networks we are concerned with. For example, the set Ω_N of permutations that can be realized by an Omega network is

$$\Omega_N = \sigma E \sigma E \cdots \sigma E = (\sigma E)^n \quad (n = \lg(N)). \quad (5)$$

In writing this expression for repeated Shuffle/Exchanges we imply a left-to-right composition of permutations, so that $\pi_1 \pi_2(x) = \pi_2(\pi_1(x))$. This possibly confusing convention is chosen to make our permutations conform to the traditional left-to-right flow of data through network drawings; so $\Omega_{16} = \sigma E \sigma E \sigma E \sigma E$ corresponds directly to Fig. 1. This formalism is useful in that it gives us an algebraic grasp on things. Notice that we can derive the expression for those permutations realized by the inverse Omega network:

$$\Omega_N^{-1} = (\Omega_N)^{-1} = ((\sigma E)^n)^{-1} = ((\sigma E)^{-1})^n = (E^{-1} \sigma^{-1})^n$$

and since $E^{-1} = E$,

$$\Omega_N^{-1} = (E \sigma^{-1})^n. \quad (6)$$

Let C_N be the set of permutations realizable by Pease's indirect binary n -cube, and R_N be those realizable by the R-network.

and

$$C_N = E \beta_{(2)} E \beta_{(3)} E \cdots E \beta_{(n)} E \sigma^{-1} \quad (7)$$

$$R_N = E \sigma_{(n)}^{-1} E \sigma_{(n-1)}^{-1} E \cdots E \sigma_{(2)}^{-1} E \rho. \quad (8)$$

Formalizing the claims of Section I, we argue that the following statement is true, establishing the functional equivalence of the networks.

Theorem 1: $\Omega_N^{-1} = C_N = R_N$.

Proof: The proof is strictly manipulative. To facilitate its execution we note the following identities, which we state without proof:

$$\rho^{-1} = \rho \quad (9a)$$

$$\sigma_{(k)}^{-1} = \sigma_{(k)}^{k-1} \quad k = 1, \cdots, n \quad (9b)$$

$$\beta_{(k)}^{-1} = \beta_{(k)} \quad k = 1, \cdots, n \quad (9c)$$

$$\sigma \rho = \rho \sigma^{-1} \quad (9d)$$

$$\beta_{(1)} \cdots \beta_{(k)} = \sigma_{(k)} \quad k = 1, \cdots, n \quad (9e)$$

$$\sigma_{(1)} \cdots \sigma_{(k)} = \rho_{(k)} \quad k = 1, \cdots, n \quad (9f)$$

where

$$\rho_{(k)}([x_n \cdots x_1]) = [x_n \cdots x_{k+1} x_1 x_2 \cdots x_{k-1} x_k].$$

To prove $\Omega_N^{-1} = C_N$, we must show expressions (6) and (7) are equivalent. We start this by noting that

$$\sigma^{-1} E = \beta_{(2)} E \beta_{(2)} \sigma^{-1},$$

an easily verified identity. Thus we have

$$\begin{aligned} \Omega_N^{-1} &= (E \sigma^{-1})^n = E(\beta_{(2)} E \beta_{(2)} \sigma^{-1}) \sigma^{-1} (E \sigma^{-1})^{n-2} \\ &= E \beta_{(2)} E \beta_{(2)} \sigma^{-2} (E \sigma^{-1})^{n-2}. \end{aligned}$$

Now it is also true that

$$\beta_{(2)} \sigma^{-2} E = \beta_{(3)} E \beta_{(3)} \beta_{(2)} \sigma^{-2}$$

so, by substituting again,

$$\Omega_N^{-1} = E\beta_{(2)}E\beta_{(3)}E\beta_{(3)}\beta_{(2)}\sigma^{-3}(E\sigma^{-1})^{n-3}.$$

Since by induction we can show

$$\beta_{(k)}\beta_{(k-1)}\cdots\beta_{(2)}\sigma^{-k}E = \beta_{(k+1)}E\beta_{(k+1)}\beta_{(k)}\cdots\beta_{(2)}\sigma^{-k}$$

we obtain

$$\Omega_N^{-1} = E\beta_{(2)}E\beta_{(3)}E\cdots E\beta_{(n)}E(\beta_{(n)}\cdots\beta_{(2)}\sigma^{-(n-1)})\sigma^{-1}.$$

Inverting (9e) and using (9c) and (9b) we get

$$\beta_{(n)}\cdots\beta_{(2)}\sigma^{-(n-1)}\sigma^{-1} = \sigma^{-1}\sigma^{-(n-1)}\sigma^{-1} = \sigma^{-1},$$

and by comparison with (7) we conclude $\Omega_N^{-1} = C_N$.

The proof that $\Omega_N^{-1} = R_N$ is similar, except that we make use of

$$\begin{aligned}\sigma^{-1} &= \sigma_{(n-1)}^{-1}E\sigma_{(n-1)}\sigma^{-1} \\ \sigma_{(n-1)}\sigma^{-2}E &= \sigma_{(n-2)}^{-1}E\sigma_{(n-2)}\sigma_{(n-1)}\sigma^{-2} \\ \sigma_{(2)}\cdots\sigma_{(n-1)}\sigma^{-(n-1)}E &= \sigma_{(2)}^{-1}E\sigma_{(2)}\cdots\sigma_{(n-1)}\sigma^{-(n-1)}\end{aligned}$$

and apply (9b) with (9f), using $k = n$ in both cases. Equivalence of Ω_N^{-1} and R_N follows exactly as above.

We have established now that the three networks are functionally equivalent, i.e., that they all realize the same set of permutations. Actually one can go further and show that the networks are isomorphic, or *topologically* equivalent, in the sense that all three are only different drawings of the same network. This can be shown directly by noting that, in the k th stage of *each* network, the only interchanges that may be made are exactly those of the form $(xx \oplus 2^k)$ where $0 \leq x \leq N-1$ (see Figs. 1-3). This statement may be proved easily by induction on the number of stages, using the definitions (6)-(8). Thus the control setting for realizing a given connection on one network is just a permutation of the control setting for another network; the exact relationship between control settings for any given exchange stage is given by a permutation involving shuffles and butterflies, which can be constructed directly from the proof of Theorem 1.

What is important now is that we have established a convenient formalism for working on networks which is useful for analyzing things besides "permuting power." In the proof of Theorem 1 we were quite vague about the precise function of "E"—it was simply a set which made identities true. If we generalize the formalism and think of E as being a class of data manipulation operations (instead of just permutations), and think of strings like (5)-(8) as being "programs" (like APL programs, but read left to right of course), then we can still find identities like those in Theorem 1 and prove the equivalence of various programs.

Thus the above analysis can be applied to the study of algorithm structures, provided we can somehow relate a known structure to one of the three networks above or something like them. A case in point is the FFT algorithm, a widely used algorithm for computing the discrete Fourier transform of a set of N data points. Introductory material may be found in [3]; we concern ourselves here with the radix-2 form of the algorithm and assume N is a power of 2, but of course the results generalize for more general condi-

tions. When viewed as a network the traditional algorithm may be written as

$$\begin{aligned}\text{FFT} &= \beta_{(n)}W\beta_{(n)}^{-1}\beta_{(n-1)}W\beta_{(n-1)}^{-1}\cdots\beta_{(1)}W\beta_{(1)}^{-1}\rho \\ &= \beta_{(n)}W\beta_{(n)}\beta_{(n-1)}W\beta_{(n-1)}\cdots\beta_{(1)}W\beta_{(1)}\rho\end{aligned}\quad (10)$$

where the W operators are columns of $N/2$ two-input/two-output "multiply-add" units, very much like the exchange operators discussed above (see Fig. 4). Because we are concerned with the gross structure of the algorithm, we ignore for the moment that the multiply-add units in each W box involve varying powers of a complex root of unity used in the transform; as long as we preserve the topological properties of the network which evaluates the FFT, then these powers still exist and can be determined.

If we invert (7), apply (9c) and replace all occurrences of E with its topological equivalent W we obtain

$$C_N^{-1} = \sigma_{(n)}W\beta_{(n)}W\cdots W\beta_{(2)}W. \quad (11)$$

So, if we propagate the easily verified identity

$$\sigma_{(k)}W\beta_{(k)} = \beta_{(k)}W\beta_{(k)}\sigma_{(k-1)}$$

from left to right in expression (11), we find

$$C_N^{-1} = \beta_{(n)}W\beta_{(n)}\beta_{(n-1)}W\beta_{(n-1)}\cdots\beta_{(2)}W\beta_{(2)}W. \quad (12)$$

Direct comparison with (10) yields

$$\text{FFT} = C_N^{-1}\rho. \quad (13)$$

Now we can apply Theorem 1. We discover immediately that

$$\text{FFT} = \Omega_N\rho = (\sigma W)^n\rho, \quad (14)$$

which indicates that the FFT algorithm can be implemented on a network which uses only shuffles and a bit reversal at the end. This is Pease's result [9]. We also find

$$\text{FFT} = R_N^{-1}\rho = (\rho W\sigma_{(2)}W\sigma_{(3)}W\cdots W\sigma_{(n)}W)\rho. \quad (15)$$

The importance of this result is that it leads to an algorithm that uses *no* bit reversal—i.e., it leads to an FFT algorithm which produces its outputs in the correct order. If we rewrite (15) using (9d) as

$$\text{FFT} = \sigma(\rho\sigma)\sigma_{(1)}W\sigma_{(2)}W\cdots W\sigma_{(n)}W\rho \quad (16)$$

and then propagate the $(\rho\sigma)$ in (16) to the right using the relationship

$$(\rho\sigma)\sigma_{(k)}W = \sigma^{-1}\sigma_{(n-k+1)}W(\rho\sigma),$$

we obtain

$$\text{FFT} = \sigma_{(n)}W\sigma^{-1}\sigma_{(n-1)}W\sigma^{-1}\cdots\sigma_{(2)}W\sigma^{-1}. \quad (17)$$

The algorithm structure corresponding to (17) is shown in Fig. 5. That the FFT could be computed without bit reversal has been known for some time (cf., [3, p. 1671], where Stockham is credited with having developed a procedure for doing it), but the butterfly/bit reversal algorithm has become standard since fast methods for computing bit reversals are known [12], and since both the butterfly and bit reversal operations can be applied to the array *in place*, i.e., no auxiliary storage is necessary to hold results from one stage

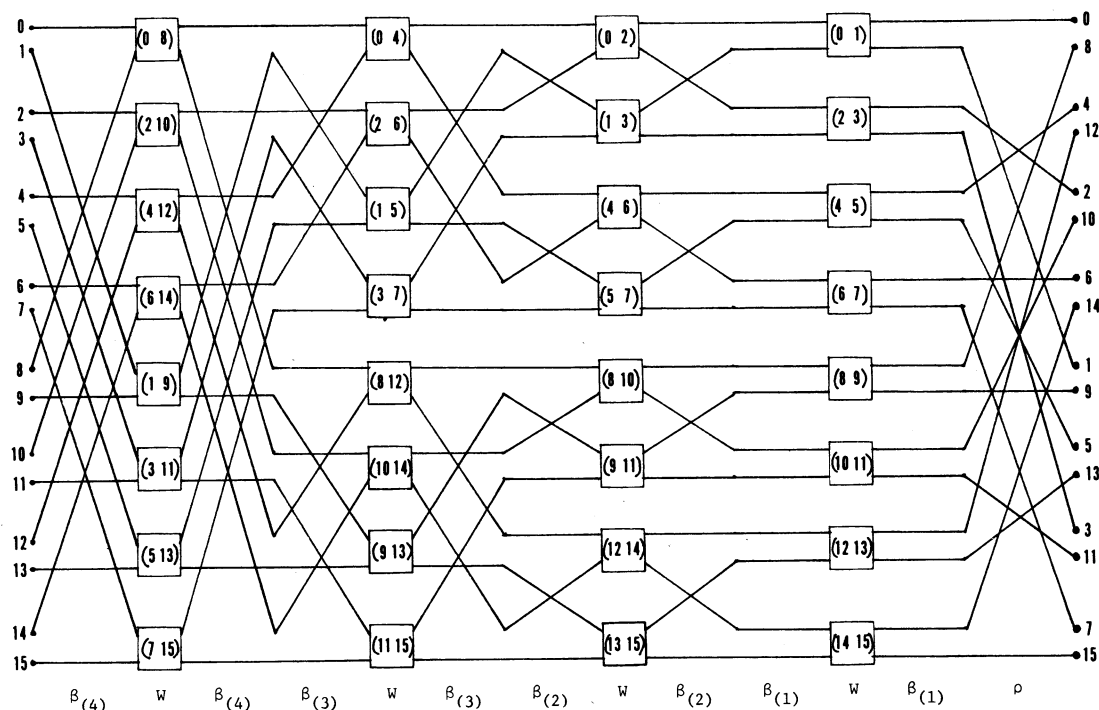


Fig. 4. Traditional FFT algorithm structure.

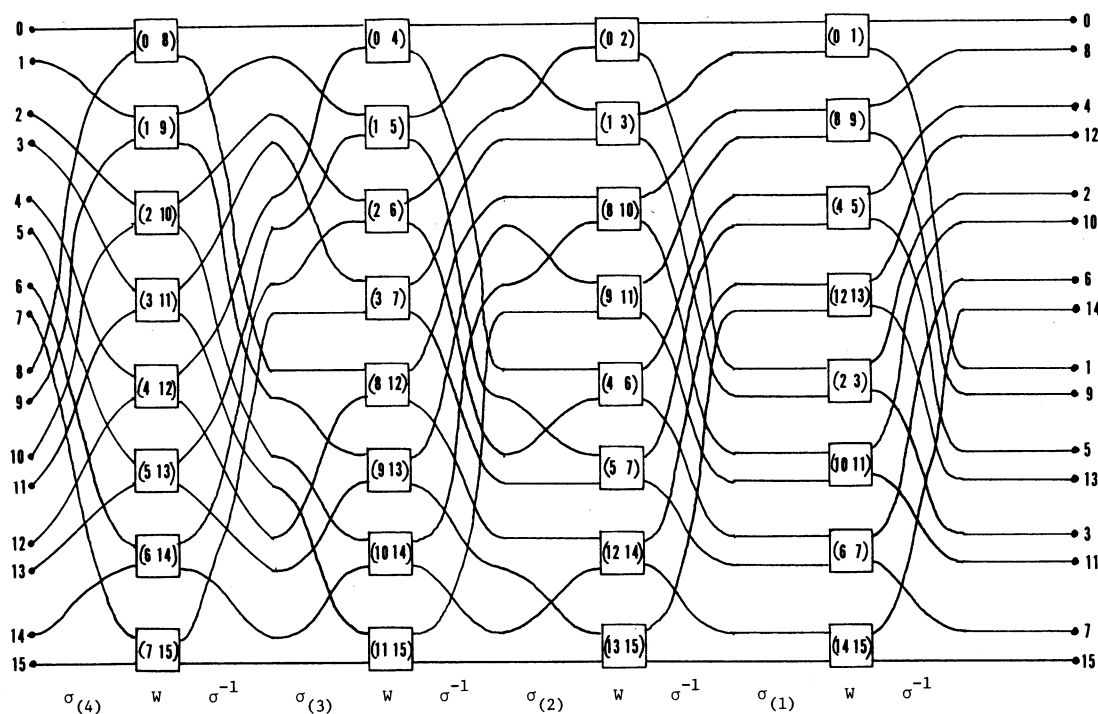


Fig. 5. FFT algorithm without bit reversal.

of the FFT to the next, as *would* be required by a serial algorithm based on expression (17). The point is, however, that on some machine architectures the traditional algorithm may not work out to be the best. If a machine is equipped to implement σ_k for all k but cannot handle ρ efficiently, then (17) is a better algorithm for that machine.

It should be pointed out that the Stockham "decimation

in time" FFT algorithm (17) may be implemented extremely efficiently on a serial machine if one does not mind using twice as much storage as the traditional FFT. The Unshuffle/Shuffle operations *do* necessitate the existence of a work array in this algorithm if it is to be coded reasonably; but great savings come in the following observation: in the usual FFT algorithm, at stage k ($k = 1, \dots, n$) one

"butterflies" the values x_i and x_{i+2^n-k} by forming

$$\begin{bmatrix} x_i \\ x_{i+2^n-k} \end{bmatrix} \leftarrow \begin{bmatrix} x_i + W^p x_{i+2^n-k} \\ x_i - W^p x_{i+2^n-k} \end{bmatrix}$$

where $W = \exp(-2\pi j/N)$, a primitive root of unity, and, if $i = [i_{n-1} \cdots i_0]$ in bit notation, then p is the integer with bit notation

$$p = [i_{n-k} i_{n-k+1} \cdots i_{n-1} 0 \cdots 0]$$

so at the final stage p is the reverse of i [i.e., $p = \rho(i)$]. The computation of W^p is a main part of the inner loop of most FFT routines since it involves evaluating many sines and cosines, as well as bit reversals in evaluating p if the program logic is not rearranged to avoid this. The importance of (17) is that it reverses the *data* at each stage just enough that the powers p come out in increasing, nonbit-reversed order as i increases; in fact one can show that, for (17),

$$p = [i_{n-1} i_{n-2} \cdots i_{n-k} 0 \cdots 0] = i - (i \bmod 2^{n-k})$$

when the algorithm is implemented properly. A simple-minded encoding is shown in Fig. 6. Obvious economies can be made on the way the values W^p are computed, and the Shuffles and Unshuffles can be merged into the inner loop by using more complicated subscript expressions.

This section then has indicated not only that the three networks under consideration are equivalent, but also has shown generally how such networks may be analyzed for equivalence (topological equivalence or equivalence of the algorithm identified with the network flow graph). Hopefully the reader has obtained some feeling for altering networks into equivalent ones by manipulating equations involving the permutations ρ , $\beta_{(k)}$, and $\sigma_{(k)}$. There is still a great deal of room left for exploration here: among other things we can show that, like (17) above we have the "decimation in frequency" structure

$$\text{FFT} = \sigma W \sigma_{(1)} \sigma W \sigma_{(2)} \cdots \sigma W \sigma_{(n-1)} \sigma W \sigma^{-1} \quad (18)$$

and there are certainly other formulations. Also, although the connections ρ , $\beta_{(k)}$, and $\sigma_{(k)}$ seem very "natural" and, when composed, are capable of generating any permutation of the index bits, there may be other connections which perform "better" for a particular algorithm than these, in the sense that fewer of them or fewer compositions of them are needed to construct a network which executes the algorithm. For example, if there were a connection π such that $\text{FFT} = (\pi W)^n \pi$, then π would be an extremely good connection to have in a Shuffle/Exchange-type network on a multiprocessor which was intended to evaluate FFT's. (Unfortunately, if π is to be a permutation on the bits of the indices (as are the shuffle and bit reversal, for example) then it is not hard to show that no such π exists for $N > 4$. Any permutation of this type that can handle the FFT must be a cycle of length n , so $\pi^n = 1$). Therefore, the architect of a new multiprocessor with a Shuffle/Exchange-type network should consider which connections will provide him with the most cost-effective switching capability for the programs to be run on the machine, and include all of them—so the Shuffle/Exchange will comprise more connections than just

```

W = exp (-2πj/N)
for k = 1 to n /* lg (N) stages */
begin
  execute σ(n-k+1) on data
  for i = 0 to (N/2) - 1 /* (N/2) 2 × 2 elements/stage */
    begin
      p = i - (i mod 2n-k)
       $\begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix} \leftarrow \begin{bmatrix} x_{2i} + W^p x_{2i+1} \\ x_{2i} - W^p x_{2i+1} \end{bmatrix}$ 
    end
    execute σ-1 on data
  end
end

```

Fig. 6. Simple FFT algorithm without bit reversal.

a shuffle. An FFT processor will probably want to include both σ and ρ ; Gajski has shown [4] that a processor which solves linear recurrences would work best if $\sigma_{(k)}$ for $k = 2, \dots, n$ were all available. Note that the bit reversal, butterfly, and shuffle permutations can all be generated using only σ and $\sigma_{(2)}$, but they are not *easily* generated. The problem of generating arbitrary index-bit permutations (therefore *a fortiori* bit reversal, shuffles, etc.) has been studied in the context of bubble memories by Wong and Copper-smith [18].

III. UNIVERSALITY OF SHUFFLE/EXCHANGE-TYPE NETWORKS

In the previous section we concerned ourselves with topological properties of Shuffle/Exchange networks, and how the same network could be represented in a number of different ways. We now address the problem of determining their inherent "permuting power," or "universality" in the terminology of Siegel [14]. That is, we want to know whether we can realize arbitrary permutations of the inputs with the Shuffle/Exchange networks if multiple passes through them are allowed. Note that this characterization of the networks has really nothing to do with the way the network is drawn—by Theorem 1, we know that the inverse Omega, indirect binary n -cube, and RSN-derived networks have identical permuting capabilities although they look different.

Let S_N denote the group of all permutations on N lines. We now ask the question: what is the smallest value of k such that

$$S_N \subseteq (\Omega_N)^k,$$

i.e., how many passes through the Omega network are necessary to ensure that any permutation can be generated? Clearly the value of k that works for Ω_N will also work for Ω_N^{-1} , R_N , and C_N , so we can restrict our attention to Ω_N here and answer the question for all these networks. One might also ask the smallest value of l such that

$$S_N \subseteq (\sigma E)^l \quad (20)$$

but clearly $(k - l/(\lg(N))) < 1$ so we have $l = k \lg(N)$ approximately, and since the behavior of the Omega networks is simpler to analyze than that of bare Shuffle/Exchanges, we will only derive bounds on k here. It would be interesting if we could show $l = k \lg(N)$ exactly.

A word about why we are interested in (19) is in order. We are concerned with the ability of our interconnection

network (in this case, an Omega network used iteratively) to permute the input values to output lines, as would be necessitated by processor-memory interaction in a multiprocessor architecture. It is true, in a multiprocessor environment like this the network will be typically requested to implement connections between inputs and outputs that are *not* perfect permutations. For example, two processors might reference the same memory module, or one processor might broadcast data to several memory modules. Temporarily we restrict our attention to S_N and (19), and address the problem of generalized connections immediately afterwards (the extension is easy).

A simple cardinality argument shows $k = 1$ is impossible in (19) for $N > 2$, since

$$\|S_N\| = N! \sim (\sqrt{2\pi})2^{(N+1/2)\lg(N) - N\lg(e)}$$

and

$$\|\Omega_N\| = 2^{(N/2)\lg(N)}.$$

However $k = 2$ cannot be immediately rejected, since it is possible that $(\Omega_N)^2$ contains enough elements. In fact, it is easy to show that $S_4 \subseteq (\Omega_4)^2$, and a computer program run on an IBM 360/75—requiring 15 min of CPU time to evaluate all 16 million elements in $(\Omega_8)^2$ —demonstrated that $S_8 \subseteq (\Omega_8)^2$ (moreover, every permutation in S_8 could be realized in at least 288, and at most 776, ways). It is tempting to conclude that $k = 2$ makes (19) true in general, since there is no evidence to contradict this. However, it seems very difficult to prove 2 passes is always enough, although we will show here that $k = 3$ is sufficient. The refinement of this bound down to $k = 2$ remains a tantalizing possibility.

It is clear that since Batcher's bitonic sorting can be implemented on a Shuffle/Exchange architecture in $(\lg(N))^2$ stages [15], [14], we have $k = \lg(N)$ as an upper bound for (19). Thus we have established the limits $2 \leq k \leq \lg(N)$. The following theorem improves the upper bound considerably.

Theorem 2: Any permutation can be realized with $\min(6, \lg(N))$ Ω_N -passes; i.e.,

$$S_N \subseteq (\Omega_N)^{\min(6, \lg(N))}.$$

Proof: This nonintuitive result (which is not the best possible) is established in a sequence of lemmas. As always we assume N is a power of 2.

Lemma 1: $S_N \subseteq \Omega_N^{-1}\Omega_N$.

Proof: Hall's theorem on systems of distinct representatives can be used to show that a rearrangeable switching network (RSN) can realize any permutation of its inputs ([2, pp. 84–88]). Therefore, $S_N \subseteq R_N R_N^{-1}$. Now apply Theorem 1.

Lemma 2: $\Omega_N^{-1} = \rho\Omega_N\rho$.

Proof: This follows immediately from the fact that $\rho\sigma E = E\sigma^{-1}\rho$, so we can propagate the leftmost ρ above rightward using (5) and wind up with (6), which gives Ω_N^{-1} .

Lemma 3: The bit reversal permutation ρ can be realized in two Ω_N -passes, i.e., $\rho \in (\Omega_N)^2$.

Proof: This is a corollary of the work of Pease [11].

Notice that

$$\rho([x_n \cdots x_1]) = P[x_n \cdots x_1]^t$$

where t denotes transpose and P is the binary matrix

$$P = \begin{pmatrix} & & & & 1 \\ & & & 1 & \\ & & \cdot & & \\ & & & \cdot & \\ & 1 & & & \\ 1 & & & & \end{pmatrix}_{n \times n}$$

Now P can be decomposed as $P = LUL$ using modulo 2 arithmetic where L and U are the binary matrices:

$$L = \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \cdot & & \\ & & & \cdot & \\ & 1 & & & 1 \\ 1 & & & & 1 \end{pmatrix}_{n \times n}$$

$$U = \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & \cdot & & \\ & & & \cdot & \\ & & & & 1 \\ 1 & & & & 1 \end{pmatrix}_{n \times n}$$

Pease has shown that any permutation $y = \pi(x)$ such that

$$y = LUX$$

where L is a lower triangular and U a unit upper triangular $n \times n$ matrix can be realized with a single Ω_N pass. Therefore since

$$\rho(x) = Px = LU(Lx)$$

ρ can be realized in two passes, or $\rho \in (\Omega_N)^2$. In fact we have $\rho(x) = \omega_2(\omega_1(x))$ where

$$\omega_1([x_n \cdots x_1]) = [x_n x_{n-1} \cdots x_{n/2} \cdot (x_{n/2} \oplus x_{n/2+1}) \cdots (x_1 \oplus x_n)] \quad (21)$$

and

$$\omega_2([x_n \cdots x_1]) = [(x_n \oplus x_1)(x_{n-1} \oplus x_2) \cdots (x_{n/2+1} \oplus x_{n/2})x_{n/2} \cdots x_n] \quad (22)$$

(assuming that n is even; the case where n is odd is similar), and it is easy to verify that both ω_1 and ω_2 are in Ω_N . For an example with $N = 8$ see Fig. 7.

Theorem 2 now follows immediately. We know

$$S_N \subseteq \Omega_N^{-1}\Omega_N \quad (\text{Lemma 1}) \quad (23)$$

$$= \rho\Omega_N\rho\Omega_N \quad (\text{Lemma 2}) \quad (24)$$

$$\subseteq (\Omega_N)^2\Omega_N(\Omega_N)^2\Omega_N = \Omega_N^6 \quad (\text{Lemma 3}). \quad (25)$$

Thus $S_N \subseteq (\Omega_N)^6$ holds for all N , but since we know $S_N \subseteq (\Omega_N)^{\lg(N)}$ for all N as well, we obtain the result stated in

the theorem. This result can be improved upon quickly, now that the principles behind it have been grasped.

Theorem 3: $S_N \subseteq (\Omega_N)^{\min(3, \lg(N))}$.

Proof: We obtain the value 3 by showing that $\rho\Omega_N \subset (\Omega_N)^2$, and by using this result immediately in (24) above. We do this as follows. Pease [11] has characterized exactly which permutations π are in Ω_N^{-1} , showing that

$$\Omega_N^{-1} = \{\pi(x) = y \mid y_i = x_i \oplus f_i(y_1, \dots, y_{i-1}, x_{i+1}, \dots, x_n)\} \quad (26)$$

where the f_i are arbitrary Boolean functions of $n-1$ variables. From this it follows immediately from Lemma 2 above that

$$\begin{aligned} \Omega_N &= \{\pi(x) = y \mid y_{n-i+1} \\ &= x_{n-i+1} \\ &\oplus f_i(y_n, \dots, y_{n-i+2}, x_{n-i}, \dots, x_1)\} \end{aligned} \quad (27)$$

and

$$\begin{aligned} \rho\Omega_N &= \{\pi(x) = y \mid y_{n-i+1} \\ &= x_i \oplus f_i(y_n, \dots, y_{n-i+2}, x_{i+1}, \dots, x_n)\}. \end{aligned} \quad (28)$$

We show that every permutation π in $\rho\Omega_N$ can be realized as a sequence of two permutations in Ω_N , where in fact the first permutation is just ω_2 defined in (22). Every permutation π in $\rho\Omega_N$ determines a unique set of Boolean functions $\{f_i \mid i = 1, \dots, n\}$, as indicated by (28). Thus for every such π we can associate a permutation ω_π in Ω_N defined as follows: if $y = \omega_\pi(z)$, so $[y_n \dots y_1] = \omega_\pi([z_n \dots z_1])$, then

$$y_{n-i+1} = z_{n-i+1} \oplus g_i(y_n, \dots, y_{n-i+2}, z_{n-i}, \dots, z_1)$$

where the g_i are in turn defined by

$$\begin{aligned} &g_i(y_n, \dots, y_{n-i+2}, z_{n-i}, \dots, z_1) \\ &= \begin{cases} f_i(y_n, \dots, y_{n-i+2}, z_{n-i}, \dots, z_1) & \text{if } i > n/2 \\ z_i \oplus f_i(y_n, \dots, y_{n-i+2}, (z_{n-i} \oplus z_{i+1}), \dots, \\ \cdot (z_{n/2} \oplus z_{n/2+1}), z_{n/2}, \dots, z_1) & \text{if } i \leq n/2 \text{ and } n \text{ is even} \\ z_i \oplus f_i(y_n, \dots, y_{n-i+2}, (z_{n-i} \oplus z_{i+1}), \dots, \\ \cdot (z_{\lfloor n/2 \rfloor + 1} \oplus z_{\lfloor n/2 \rfloor - 1}), z_{\lfloor n/2 \rfloor}, \dots, z_1) & \text{if } i \leq n/2 \text{ and } n \text{ is odd.} \end{cases} \end{aligned}$$

With these definitions it is simple to verify $\pi(x) = \omega_\pi(\omega_2(x))$ is true for all x and π . Thus we have shown $\rho\Omega_N \subset (\Omega_N)^2$. Now, since S_N is a group we know $S_N = \rho^{-1}S_N$. By (24), therefore, $S_N \subseteq \Omega_N \rho\Omega_N$.

Thus, it follows immediately that

$$S_N \subseteq \Omega_N(\rho\Omega_N) \subset \Omega_N^3 \quad (29)$$

is true for all N , giving the bounds $2 \leq k \leq 3$ on the least value of k that will satisfy (19). Again, the construction used

for $k = 3$ seems inefficient when one takes into consideration that one of the three Omega passes realizes only the constant permutation ω_2 —an apparently wasteful situation. However, it is not clear that $k = 2$ will suffice, and it remains an open question to determine precisely which k is minimal for any value $N > 8$.

Theorems 2 and 3 lead to some interesting derivatives. Define the *F-network* to be a network which realizes the set of permutations

$$F_N = \Omega_N \rho. \quad (30)$$

The first half of the RSN studied by Beneš [2] and Opferman and Tsao-Wu [8] is an example of an *F-network*. Then we can make three observations: First, (14) tells us that the *F-network* can be used to implement FFT's. Second, we have

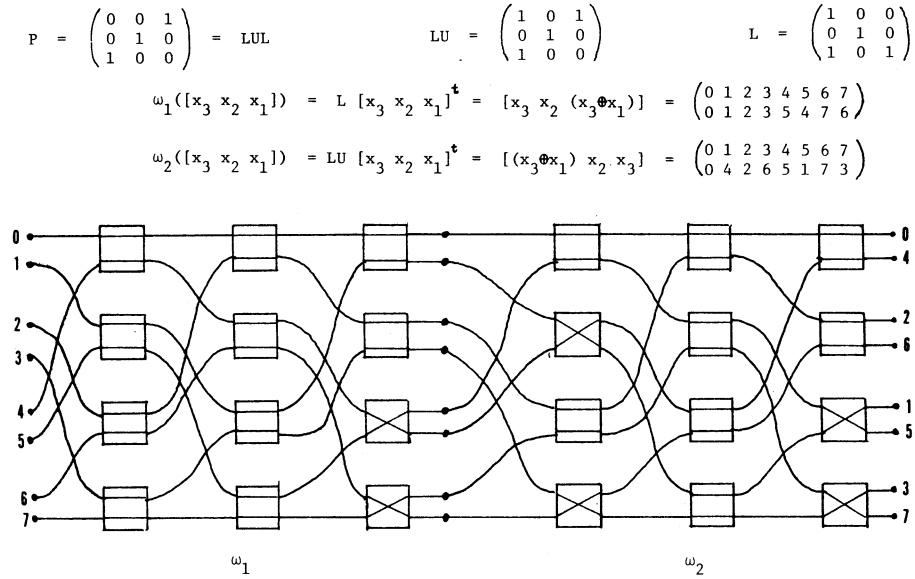
$$S_N \subseteq (F_N)^2 \quad (31)$$

since $S_N = \rho S_N \rho \subset \rho(\rho\Omega_N \rho\Omega_N)\rho = (\Omega_N \rho)^2$. Thus the analogue of (19) for *F-networks* is always satisfied by $k=2$. Third, and most surprisingly, noting that $F_N^{-1} = (\rho\Omega_N)^{-1} = \Omega_N^{-1}\rho = (\rho\Omega_N \rho)\rho = \rho\Omega_N$, we find

$$F_N = F_N^{-1} \quad (32)$$

and the network turns out to be topologically equivalent to itself when run "backwards." It remains to be seen whether these properties have any useful implications; disappointingly, the *F-network* is incapable of realizing the identity permutation.

The result (29) is interesting for the following reason. When the author originally formulated the question of finding the least value of k satisfying (19) he was confident that the answer would be $k = \lg(N)$, which would have suggested that Batchier's algorithm is essentially optimal for sorting or realizing arbitrary permutations on a Shuffle/Exchange network. However, (29) implies that the optimality of Batchier's method is not so easy to demonstrate. In fact, Lemmas 1–3 and Theorem 3 say that if we can find a rapid control algorithm for the RSN, then we can realize arbitrary permutations in a small constant number of Ω_N -passes, significantly less than the $\lg(N)$ passes required by bitonic sorting. (Unfortunately, the best known RSN control algorithm at the moment is related to the "Looping algorithm" of Opferman and Tsao-Wu [8], and takes $O(N)$ steps to execute.) This brings the complexity of controlling the network under focus as the main question concerning us—notice that the Batchier algorithm uses a strictly *local* control process (pairwise comparisons), while the RSN control algorithm is strictly *global*. The Batchier algorithm thus sacrifices a number of Ω_N -passes for control simplicity, whereas the RSN algorithm takes the opposite tack. We ask: is there a *semiglobal* control strategy lying between the strictly local Batchier and strictly global RSN algorithms which uses both a modest amount of control time and a limited number of Ω_N -passes? Wen [17] has done some

Fig. 7. Demonstration that ρ is in $(\Omega_8)^2$.

preliminary work indicating that this is probably the case, at least for the average number of passes required by random permutations. Results here could have great impact on the design of processor-memory interconnection networks.

Finally we bring up, as promised, the problem of making arbitrary connections using Shuffle/Exchange-type networks. Note first of all that if m inputs all desire to come out on the same output line, then we have no choice but to delay $(m - 1)$ of them, and let these through one at a time after the first one has reached its destination. Thus at least $m \Omega_N$ -passes are needed to transfer all the data through the network, and there is little else to say. If, however, one input desires to be "broadcasted" (in the sense of [7]) to m outputs, then it is still possible that a small constant number of Ω_N -passes could suffice. Let T_N be the set of arbitrary connections between all N inputs and outputs; then clearly $S_N \subset T_N$ and

$$\|T_N\| = N^N. \quad (33)$$

Unfortunately the RSN is not powerful enough to cover all of T_N , or even arbitrary broadcast patterns, so we do not have

$$T_N \subseteq \Omega_N^{-1} \Omega_N$$

and Theorems 2 and 3 do not hold for generalized connections. Fortunately, the problem has been studied by Thompson in [16]. He defines a generalized connection network (GCN) with the network structure/set of connections

$$\begin{aligned} G_N &= B\beta_{(2)}B\beta_{(3)}B \cdots B\beta_{(n)}B\beta_{(n-1)}B \\ &\quad \cdots B\beta_{(2)}B \cdots B\beta_{(n-1)}B\beta_{(n)}B \cdots B\beta_{(2)}B \\ &= C_N \sigma(B\beta_{(n-1)}B \cdots B\beta_{(2)}B \\ &\quad \cdots B\beta_{(n-1)}B) \sigma^{-1} C_N^{-1} \end{aligned} \quad (34)$$

where C_N denotes (7) with E 's replaced by B 's, and B is the set of all connections realizable with a column of exchange elements when broadcasting is permitted. Following (4) we can formally define

$$\begin{aligned} B &= \{\text{connections } b \mid \text{for every } 0 \leq x \\ &\quad \leq N - 1 \text{ with } x \text{ even} \\ &\quad \text{either } b(x) = x \text{ and } b(\hat{x}) = \hat{x} \\ &\quad \text{or } b(x) = \hat{x} \text{ and } b(\hat{x}) = x \\ &\quad \text{or } b(x) = b(\hat{x}) = x \\ &\quad \text{or } b(x) = b(\hat{x}) = \hat{x}\}. \end{aligned} \quad (35)$$

Thus B is a set with $4^{N/2} = 2^N$ elements. The operation of Thompson's network is as follows. The first half is a "generalizer," a network which passes each input i to m_i different outputs, where $\sum m_i = N$ and $m_i \geq 0$. This half merely creates the right number of copies of each input necessary to satisfy the desired connection. The second half of the network is a "connector," which permutes its inputs to their desired outputs. Thus this second half need only realize the connections in S_N , and all the B 's in the right half of (34) could be replaced by E 's.

Now, (34) may be easily padded to show

$$G_N \subseteq (C_N C_N^{-1})(C_N C_N^{-1})$$

where the first half of the right-hand side comprises the generalizer and the second half the connector. Thompson has shown $T_N \subseteq G_N$; therefore using the results above (with E 's replaced by B 's) we find

$$\begin{aligned}
T_N &\subseteq G_N \\
&\subseteq (C_N C_N^{-1})(C_N C_N^{-1}) \\
&= (\Omega_N^{-1} \Omega_N)(\Omega_N^{-1} \Omega_N) \quad (\text{Theorem 1}) \\
&\subseteq (\Omega_N^{-1} \Omega_N)(\Omega_N^3) \quad (\text{Theorem 3}) \\
&\subseteq (\rho \Omega_N \rho \Omega_N)(\Omega_N^3) \quad (\text{Lemma 2}) \\
&\subseteq (\rho \Omega_N^3)(\Omega_N^3) \quad (\text{Lemma 3}) \\
&\subseteq (\rho \Omega_N^6).
\end{aligned}$$

Since $T_N = \rho^{-1} T_N$, we have shown that arbitrary broadcast patterns can be realized in 6 Ω_N -passes, given that the exchange elements are equipped to make upper or lower broadcasts. Alternatively, we can use the same argument to establish this result using 4 F -network passes. Once again, whether these pass counts can be further refined is an open question.

IV. CONCLUSION

This paper has analyzed properties of several Shuffle/Exchange-type networks of current interest in the design of parallel architectures: the inverse Omega network, the Indirect binary n -cube, and the R -network (a network derived from the 3-stage rearrangeable switching networks studied by Clos and Beneš). The algebraic study of several basic connections—shuffles, butterflies, and bit-reversals—lead to easy demonstrations of the functional and topological equivalence of the three networks above. It also led to a demonstration of the equivalence of various FFT algorithms, one using only full shuffles and a final bit reversal, one using only various shuffles and *no* bit reversal.

With this established, the "universability" of these networks—their ability to realize arbitrary permutations if multiple passes through them be permitted—was examined. It was shown that 3 passes through these networks is sufficient to generate any *permutation* (two passes being necessary), and 6 passes is sufficient to generate any *connection*, of the input to the output lines. An interesting network, called the F -network, was introduced and shown to improve these pass counts to 2 and 4, respectively, the minimal number of passes necessary. These pass count results have possible implications on the control of Shuffle/Exchange-type networks for parallel computation applications.

ACKNOWLEDGMENT

The author is totally indebted to Dr. D. Kuck, without whom this work would never have been done. He is also very grateful to D. H. Lawrie for several stimulating discussions, and to M. C. Pease of SRI, Inc., for various important technical comments which improved the quality of the paper.

REFERENCES

- [1] K. E. Batcher, "Sorting networks and their applications," in *Proc. AFIPS SJCC*, vol. 32, pp. 307–314, 1968.
- [2] V. E. Beneš, *Mathematical Theory of Connecting Networks and Telephone Traffic*. New York: Academic Press, 1965.
- [3] W. T. Cochran *et al.*, "What is the fast Fourier transform?" *Proc. IEEE*, vol. 55, pp. 1664–1674, Oct. 1967.
- [4] D. D. Gajski, "Processor arrays for computing linear recurrence systems," presented at Int. Conf. on Parallel Processing, Aug. 23–25, 1978.
- [5] T. Lang, "Interconnections between processors and memory modules using the shuffle-exchange network," *IEEE Trans. Comput.*, vol. C-25, pp. 496–503, May 1976.
- [6] T. Lang and H. S. Stone, "A shuffle-exchange network with simplified control," *IEEE Trans. Comput.*, vol. C-25, pp. 55–65, Jan. 1976.
- [7] D. H. Lawrie, "Access and alignment of data in an array processor," *IEEE Trans. Comput.*, vol. C-25, pp. 1145–1155, Dec. 1976.
- [8] D. C. Opferman and N. T. Tsao-Wu, "On a class of rearrangeable switching networks—Parts I and II," *Bell Syst. Tech. J.*, vol. 50, pp. 1579–1618, May–June 1971.
- [9] M. C. Pease, "An adaptation of the fast Fourier transform for parallel processing," *J. Assoc. Comput. Mach.*, vol. 15, pp. 252–264, Apr. 1968.
- [10] —, "Organization of large scale Fourier processors," *J. Assoc. Comput. Mach.*, vol. 16, pp. 474–482, July 1969.
- [11] —, "The indirect binary n -cube microprocessor array," *IEEE Trans. Comput.*, vol. C-26, pp. 458–473, May 1977.
- [12] R. J. Polge *et al.*, "Fast computational algorithms for bit reversal," *IEEE Trans. Comput.*, vol. C-23, pp. 1–9, Jan. 1974.
- [13] H. J. Siegel, "Analysis techniques for SIMD machine interconnection networks and the effects of processor address masks," *IEEE Trans. Comput.*, vol. C-26, pp. 153–161, Feb. 1977.
- [14] —, "The universality of various types of SIMD machine interconnection networks," in *Proc. 4th Ann. Symp. Computer Architecture*, Mar. 23–25, 1977.
- [15] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.*, vol. C-20, pp. 153–161, Feb. 1971.
- [16] C. D. Thompson, "Generalized connection networks for parallel processor intercommunication," *IEEE Trans. Comput.*, vol. C-27, pp. 1119–1125, Dec. 1978.
- [17] K.-Y. Wen, "Interprocessor connections—Capabilities, exploitation and effectiveness," Rep. UIUCDCS-R-76-830, Univ. of Illinois at Urbana/Champaign, Oct. 1976.
- [18] C. K. Wong and D. Coppersmith, "The generation of permutations in magnetic bubble memories," *IEEE Trans. Comput.*, vol. C-25, pp. 254–262, Mar. 1976.



D. Stott Parker, Jr. (S'76-M'79) was born in New Haven, CT, in 1952. He received the A.B. degree in mathematics from Princeton University, Princeton, NJ, in 1974, and the M.S. and Ph.D. degrees in computer science in 1976 and 1978, respectively, from the University of Illinois at Urbana-Champaign.

Currently he is an Assistant Professor with the Department of Computer Science at the University of California, Los Angeles. His main interests include database theory, distributed processing, algorithms, and architecture.

Dr. Parker is a member of Sigma Xi and the Association for Computing Machinery.